Pipeline Done Conditions Reference Specification

This document defines the exact, codebase-level mathematical and on-disk criteria that the pipeline orchestrator evaluates to classify each processing phase as complete (**Done**).

Initialization and Pre-Flight Gates

Step 0: checkin

- **Logical Meaning:** The active project pointer has been successfully switch-bound and the target project workspace directory tree has been physically scaffolded on disk.
- **Done Condition:** Symmetrically, the active project pointer file physically exists under the repository root, its content resolves to a valid project directory path, and the directory tree has been created on disk.
- **Status Text:** Returns the active project directory folder name.

Step 1: init (environment)

- **Logical Meaning:** The host machine software capabilities, packages, and credential configurations have been audited and passed.
- **Done Condition:** Symmetrically, the environment audit passed marker file exists inside the active project directory, and the local project marker file has been touched.
- **Status Text:** Returns `OK` if present; otherwise reports missing packages or `not yet audited`.

Step 2: ingest

- **Logical Meaning:** Technical source PDFs have been scanned, registered, and short-hashed.
- **Done Condition:** The document inventory database physically exists and contains at least one registered document record.
- **Status Text:** Returns document counts grouped by their directory Machine ID.

Step 3: define-layers (ontology)

- **Logical Meaning:** The immutable layers classification ontology has been created inside the workspace.
- **Done Condition:** The ontology database file physically exists inside the library folder.
- **Status Text:** Returns the count of registered layers.

Step 4: documentation-layers

- **Logical Meaning:** Document-level layer assignments have been proposed by the classifier and approved by the operator.
- **Done Condition:** Symmetrically, the document inventory database exists, contains at least one document, and exactly zero documents are pending layer approval.
- **Status Text:** Returns the proportion of approved documents.

Visual Ingestion, Subdivision, and Annotation

Step 5: macro-images

- **Logical Meaning:** Structural layout macro-objects (views, BOM tables, specification blocks) are discovered on-screen and drafted as coordinate regions on disk.
- **Done Condition:** The persistent drafting queue manifest physically exists inside the crops folder on-disk.
- **Status Text:** Returns a combined hybrid status detailing discovery checklist items completed, pending items, and total macro regions promoted.

Step 6: verify-drafts

- **Logical Meaning:** Proposed macro candidates are validated against image-quality rules and promoted to the canonical regions catalog.
- **Done Condition:** The macro-images step is done AND the active drafting queue manifest contains exactly zero macro-level drafts.
- **Status Text:** Returns the count of pending macro drafts, or the count of promoted macro regions in the catalog upon queue completion.

Step 7: micro-images

- **Logical Meaning:** Granular detail views and text-geometry pairs are drafted from promoted macro views.
- **Done Condition:** Symmetrically, at least one subdivision page census checklist file exists inside the page-checklists census folder on-disk.
- **Status Text:** Returns a combined hybrid status detailing macro regions subdivided, pending subdivisions, and total micro regions promoted.

Step 8: verify-drafts --micro

- **Logical Meaning:** Proposed detail view candidates are validated against image-quality rules and promoted to the canonical regions catalog.
- **Done Condition:** The micro-images step is done AND the active drafting queue manifest contains exactly zero micro-level drafts.
- **Status Text:** Returns the count of pending micro drafts, or the count of promoted micro detail regions in the catalog upon queue completion.

Step 9: review (regions) (HITL Gate 3)

- **Logical Meaning:** Bounding box coordinates and category labels for all proposed regions have been reviewed, adjusted, and approved by the operator.
- **Done Condition:** Symmetrically, the canonical regions database exists, contains at least one region, and every single region holds human approval.
- **Status Text:** Returns the proportion of approved regions.

Step 10: caption

- **Logical Meaning:** Detailed textual descriptions and visual search prompts are computed for every approved region crop.
- **Done Condition:** Every single approved region inside the regions database carries both a non-empty caption and a non-empty vision prompt.
- **Status Text:** Returns the proportion of captioned regions.

Asset Binding, Indexing, and Publishing

Step 11: ingest --images (Full Asset Binding)

- **Logical Meaning:** Approved visual crops are bound to unique, durable asset URN identifiers.
- **Done Condition:** Symmetrically, the asset inventory database exists and contains at least one finalized asset URN binding.
- **Status Text:** Returns active assets out of total approved regions, appending the count of rejected assets from the rejections audit log if present (e.g., `290/293 assets, 3 rejected`).

Step 12: asset-layers

- **Logical Meaning:** A fine-grained ontology layer has been assigned and verified for every asset.
- **Done Condition:** Symmetrically, the asset inventory database has at least one asset, and every single asset is successfully verified.
- **Status Text:** Returns assigned assets out of total approved regions, appending the count of rejected assets from the rejections audit log if present (e.g., `290/293 assigned, 3 rejected`).

Step 13: review --assets (HITL Gate 4)

- **Logical Meaning:** Asset layers and captions have been reviewed and approved by the operator.
- **Done Condition:** The asset inventory database exists, contains at least one asset, and exactly zero assets are pending approval.
- **Status Text:** Returns approved assets out of total approved regions, appending the count of rejected assets from the rejections audit log if present (e.g., `290/293 approved, 3 rejected`).

Search Indexing and Publishing

Step 14: build-index

- **Logical Meaning:** Keyword search and semantic vector databases are populated.

- **Done Condition:** Symmetrically, the index build success marker file physically exists inside the logs directory.
- **Status Text:** Lists the active searchable indexes (e.g., `FTS5 ready, ChromaDB ready`).

Step 15: verify

- **Logical Meaning:** Deep integrity check verifies SHA-256 hashes, crop files, and URN links.
- **Done Condition:** Symmetrically, the verification success marker file physically exists inside the logs directory.
- **Status Text:** Returns `OK` if present; otherwise `not verified`.

Step 16: export (Machine Catalog Publishing)

- **Logical Meaning:** Symmetrically, the published machine catalog JSON database is compiled and ready for distribution.
- **Done Condition:** Symmetrically, at least one published machine catalog file physically exists inside the library inventory directory.
- **Status Text:** Returns the number of published machines.

Step 17: query (warmup)

- **Logical Meaning:** Pre-flight hybrid searches have been executed to warm up indices and verify cited synthesizer correctness.
- **Done Condition:** Symmetrically, the warmup success marker file physically exists inside the logs directory.
- **Status Text:** Returns `warmed up`.

Step 18: chat (warmup)

- **Logical Meaning:** Symmetrically, an interactive conversational session has been launched over the hybrid retrieval indices.
- **Done Condition:** Symmetrically, the chat success marker file physically exists inside the logs directory.
- **Status Text:** Returns `warmed up`.